

Transformers:

Queremos embeddings que dependan del contexto:

Banco $\rightarrow$ institución financiera
silla

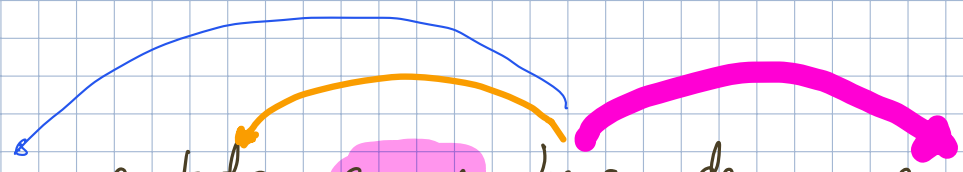
Necesitamos un modelo que "vea" todas las otras palabras en el texto y construya el embedding correspondiente:

"cuenta", "dinero" $\rightarrow$ inst. financiera

"perro", "sentado" $\rightarrow$ silla parque.

Self attention:

Había una persona sentada en un banco de un parque.



Ahora en vez de sumar los embeddings de las palabras de contexto dando un peso uniforme, queremos darle distinto peso a distintas palabras:

Word2vec: $h = \sum_{j \in \text{contexto}} 1 \cdot u_j$ atención, lo vemos a estandar.

self attention: $h_i = \sum_{j \in \text{contexto}} \alpha_{ij} u_j$



¿Cómo estimamos α_{ij} ?

Para cada palabra i , construimos 3 vectores:

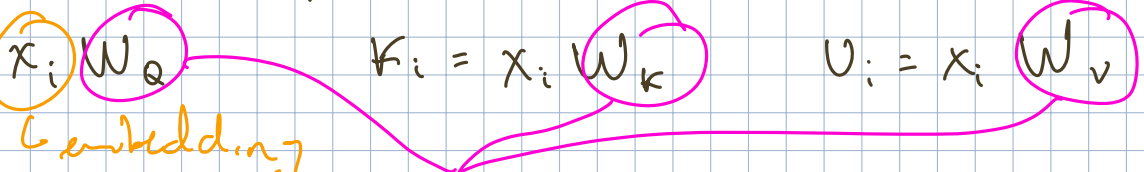
Q_i : query $\rightarrow$ Qué información necesito?

K_i : key $\rightarrow$ Qué información tengo con contexto

V_i : value $\rightarrow$ Qué info puedo transferir si se me presta atención.

$$Q_i = x_i W_Q \quad K_i = x_i W_K \quad V_i = x_i W_V$$

x_i embedding



matrices que estimamos.

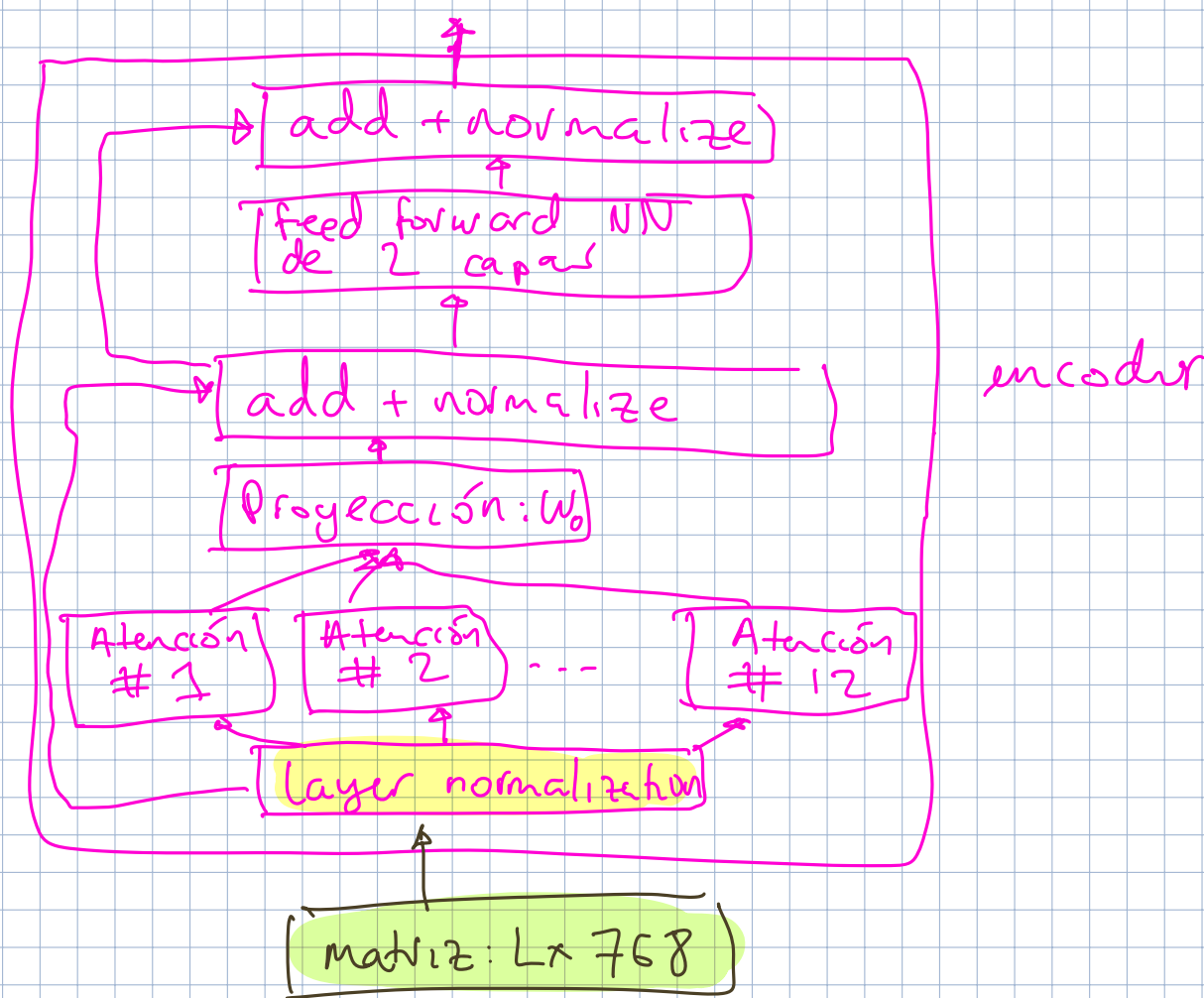
- en word2vec estábamos W , aquí W_Q, W_K, W_V .

$$\text{attention}(Q, K, V) = \underbrace{\text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right)}_{\text{nos da los } \alpha\text{'s}} \cdot V \quad \leftarrow \text{nos da el equivalente a los vectores } h \text{ en word2vec}$$

QK^T : scores de afinidad: para cada palabra qué tanto importa cada una de las otras palabras.

$\sqrt{d_k}$: normalización para estabilidad del softmax.

Encoder/decoder:



- Hiperparámetro: dimensión de embeddings: GPT 2 : 768
- Tenemos un texto: "La vaca es grande".
L tokens
- Vocabulario: tamaño 100.000

- matriz de embeddings de dimensión: 100000×768
- se seleccionan los L embeddings correspondientes al texto:
matriz de tamaño $L \times 768$
- le sumamos una matriz que codifica la posición de la palabra en el texto
 $\hookrightarrow$ positional embedding

Componentes del encoder:

1. **Layer normalization**: se normaliza cada uno de los L vectores para que tengan media 0 y varianza 1.

2. **Atención**: Estimar matrices W_Q, W_K, W_V de dimensión 768×64 $\hookrightarrow$ hiperparámetro

Q, K, V tienen dimensión $L \times 64$

$Q \cdot K^T$ dimensión $L \times L$

$\sqrt{d_k} = 8$

$\underbrace{\text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{64}}\right)}_{\text{dimensión } L \times L} \cdot \underbrace{V}_{\text{dim } L \times 64} \longrightarrow \text{dimensión } L \times 64$

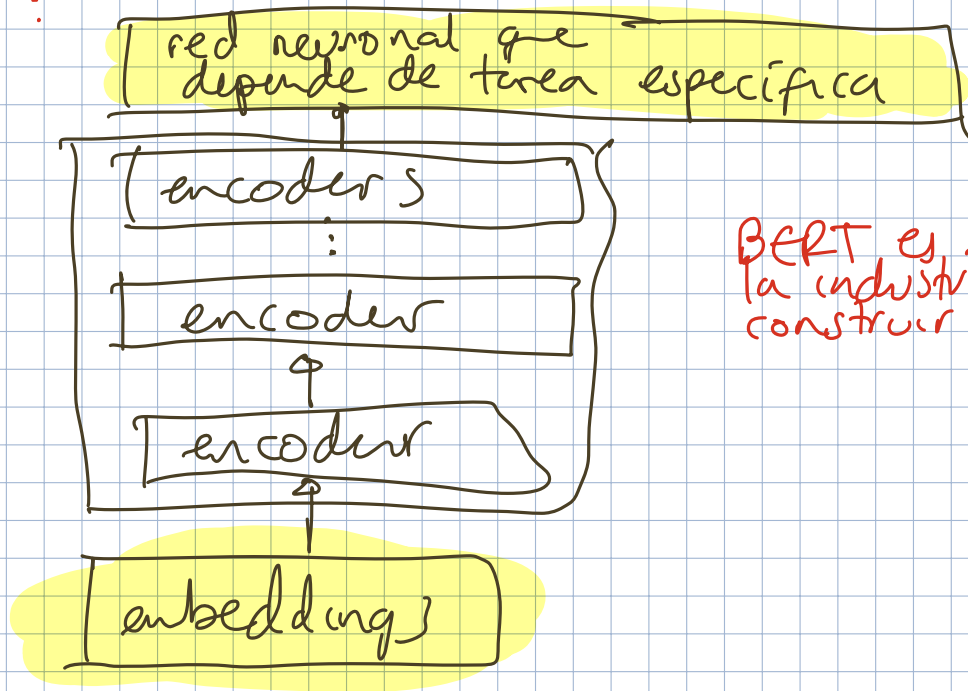
Si son 12 cabezas de atención $\Rightarrow$ tenemos 12 matrices de dimensión $L \times 64$

Concatenamos esas 12 matrices para obtener una única matriz de dimensión $L \times 768$

3. **Proyección**: multiplicamos esa matriz $L \times 768$ por una matriz W_O de dimensión 768×768 para combinar la información de las 12 cabezas de atención

4. Add y normalize: Sumar esta matriz de tamaño $L \times 768$, con la matriz inicial y normalizarlo.
5. feed forward NN: Red neuronal de 2 capas.
6. Add y normalize.

BERT:



BERT es estándar en la industria para construir embeddings.

GPT: En vez de encoders, usan decoders.

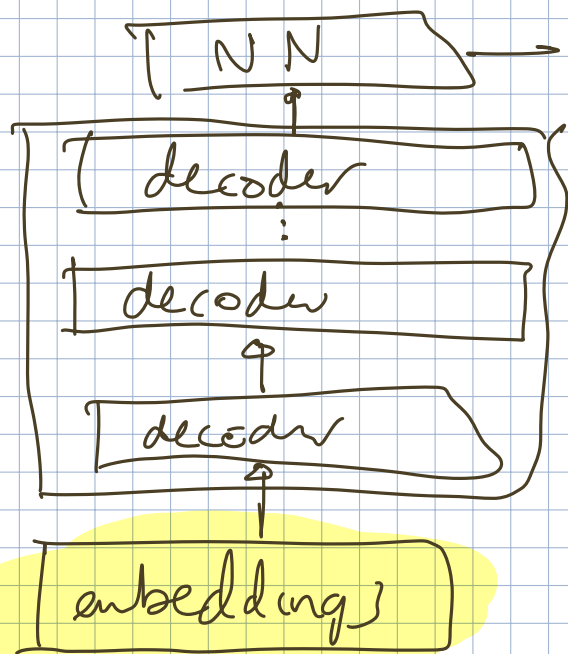
Decoders son idénticos excepto por una diferencia:

QK^T — en BERT, cada palabra puede "mirar" las palabras a la derecha o a la izquierda.
 $\Rightarrow$ BERT NO es generativo.

En un decoder:

En $Q \times K^T$, matriz cuadrada de $L \times L$, reemplazamos el triángulo superior por $-\infty$

$$Q \times K^T = \begin{pmatrix} & & \\ & \text{triángulo superior} & \\ & & \end{pmatrix}_{L \times L}$$



para predecir la siguiente palabra.